Linear Regression

Lecture notes, Machine Learning, Week 2

Regression problems

Regression problems pop up whenever we want to predict a numerical value. Common examples include predicting prices of homes or stocks, predicting the length of stay of patients in hospital, and forecasting demand for retail sales.

As a running example, suppose we wish to estimate the prices of houses in dollars from their area in square feet and their age in years. The dataset is called a training set, and each row, one sale, is called an example. The thing we are trying to predict, the price, is called the label or target. The variables the prediction is based on, area and age, are called features.

Assumptions

Linear regression is both the simplest and the most popular of the standard tools for regression, dating back to the start of the 19th century. It assumes first that the relationship between the features x and the target y is approximately linear: the conditional mean $E[Y \mid X = x]$ is a weighted sum of the features. The target may still deviate from its expected value because of observation noise.

Second, it assumes that the noise is well behaved, following a Gaussian distribution.

The model

The model predicts price = w_area * area + w_age * age + b. Here w_area and w_age are called weights and b is called the bias, offset or intercept. The weights determine the influence of each feature on the prediction. The bias is the value of the estimate when all features are zero; it is needed so the model can express every linear function of the features, not only lines that pass through the origin.

Strictly speaking the model is an affine transformation: a linear transformation of the features by a weighted sum, combined with a translation by the bias. With d features collected in a vector x, the prediction is the dot product y_hat = w^T x + b. For a whole dataset, the design matrix X has one row for every example and one column for every feature, and the predictions are y_hat = Xw + b.

The squared error loss

Loss functions quantify the distance between the real and predicted values of the target. The loss is usually nonnegative, smaller is better, and a perfect prediction has a loss of 0. For regression the most common loss is the squared error, l = 1/2 (y_hat - y)^2. The constant 1/2 makes no real difference but cancels out when the loss is differentiated.

Because of its quadratic form, large differences between estimates and targets lead to even larger contributions to the loss. This encourages the model to avoid large errors, but also makes it excessively sensitive to anomalous data. Over the whole training set the losses are averaged, and training seeks the weights and bias that minimise that average.

The analytic solution

Unlike most models, linear regression can be solved analytically. Folding the bias into the weights by appending a column of 1s to the design matrix, the problem is to minimise ||y - Xw|| ^2. The optimal weights are w* = (X^T X)^-1 X^T y.

This solution is unique only when the matrix X^T X is invertible, that is, when the columns of the design matrix are linearly independent and no feature is linearly dependent on the others. Analytic solutions are rare: requiring one would exclude almost all of deep learning.

Minibatch stochastic gradient descent

Gradient descent reduces the error iteratively, updating the parameters in the direction that lowers the loss. Computing the gradient over the whole dataset before every update is extremely slow. Stochastic gradient descent (SGD) updates after a single example, but processors multiply and add numbers much faster than they move data from memory, and some layers, such as batch normalisation, need more than one example at a time.

The solution is a minibatch: a small random sample of examples. A minibatch size between 32 and 256, preferably a multiple of a large power of 2, is a good start. In each iteration a minibatch is sampled, the gradient of its average loss is computed, multiplied by a small positive learning rate, and subtracted from the parameters.

The minibatch size and the learning rate are hyperparameters: they are set by the user rather than updated in the training loop, and can be tuned, for example by Bayesian optimisation. The quality of the result is assessed on a separate validation set.

Vectorisation for speed

When training, a whole minibatch of examples is processed at once. Adding two vectors of 10,000 elements with a Python for-loop, one element at a time, is far slower than one vectorised addition, which is orders of magnitude faster. Vectorised code is also less error prone, because it is shorter and leaves the arithmetic to the library.

The normal distribution and squared loss

The squared loss can be motivated by assuming the observations come from noisy measurements, y = w^T x + b + noise, where the noise follows a normal distribution with mean 0 and standard deviation sigma.

Maximising the likelihood of the data under this assumption is the same as minimising the negative log-likelihood, and that is the same as minimising the mean squared error. So under Gaussian noise, minimising the squared error is the maximum likelihood estimator of a linear model.

Linear regression as a neural network

Linear regression can be drawn as a single-layer neural network. The inputs are the features x1 to xd, and there is one output, o1. Every input is connected to the output, so the layer is a fully connected layer, also called a dense layer. The number of layers counts only the layers that compute, so the network has one layer.

A biological neuron receives signals from other neurons through its dendrites, weights them by synaptic strengths, sums them in the cell body, and sends the result along its axon. Modern deep learning draws more on mathematics, statistics and computer science than on neuroscience.

Summary

Linear regression fits a weighted sum of the features plus a bias by minimising the squared error, usually with minibatch stochastic gradient descent. Its key ingredients, a model, a loss function and an optimisation algorithm, are the ingredients of nearly every deep learning model.

Sources

Adapted from Dive into Deep Learning by Aston Zhang, Zachary C. Lipton, Mu Li and Alexander J. Smola (https://d2l.ai), licensed CC BY-SA 4.0. This adaptation is shared under the same licence.